SIMATS ENGINEERING

ASSESSMENT TOOL 1 – ANALYTICAL PROBLEM SOLVING

Course: Artificial Intelligence **Course Code:** CSA17

Solved Answers

Question 1: Water Jug Problem

Given: One 4-gallon jug and one 3-gallon jug. The aim is to obtain exactly 2 gallons in the 4-gallon jug.

State format: (amount in 4-gallon jug, amount in 3-gallon jug).

Step	Action	State (4-gallon, 3-gallon)
Initial	Both jugs are empty	(0, 0)
1	Fill the 3-gallon jug	(0, 3)
2	Pour the 3-gallon jug into the 4-gallon jug	(3, 0)
3	Fill the 3-gallon jug again	(3, 3)
4	Pour from the 3-gallon jug into the 4-gallon jug until the 4-gallon jug is full. Only 1 gallon fits.	(4, 2)
5	Empty the 4-gallon jug onto the ground	(0, 2)
6	Pour the remaining 2 gallons from the 3-gallon jug into the 4-gallon jug	(2, 0)

Final answer: The 4-gallon jug contains exactly 2 gallons.

Question 2: Mars Rover as an Intelligent Agent

i. Percepts: Images from cameras; distance and obstacle readings from sensors; wheel movement/position data; temperature, pressure, and other environmental readings; and information about rocks or soil from scientific instruments.

ii. Operating environment: Mars surface. It is partially observable, uncertain, dynamic over time, sequential, and mostly discrete for planning but continuous for movement. Communication with Earth has a time delay and power/resources are limited.

iii. Actions: Move forward/backward; turn; stop; avoid obstacles; take photographs; use robotic arm to collect or examine samples; operate scientific instruments; store and transmit data; choose a route and manage power.

iv. Performance evaluation: Successful arrival at target locations; useful and accurate scientific data collected; sample analysis completed; obstacles avoided; mission goals achieved; and power, time, and equipment used efficiently.

v. Suitable architecture: A model-based, goal-based agent with utility-based decision-making. A model-based component keeps track of the rover's surroundings when not everything can be sensed at once. Goal-based planning helps it reach sites and collect samples, while utility-based decisions balance scientific value, risk, time, and battery power.

Question 3: 8-Queens Problem

Problem: Place 8 queens on an 8 × 8 chessboard so that no two queens share a row, column, or diagonal.

1. Initial state: An empty chessboard.

2. State representation: Use one number for each column; the number gives the row containing that column's queen.

3. Actions: Place a queen in a safe square in the next column, ensuring it does not attack any previously placed queen.

4. Goal test: All 8 queens are placed and no two queens attack each other.

5. Path cost: Usually one unit per queen placement; every complete solution uses 8 placements.

Example solution (column → row): C1→R1, C2→R5, C3→R8, C4→R6, C5→R3, C6→R7, C7→R2, C8→R4.

Check: All row numbers are different. For each pair of queens, the row difference is not equal to the column difference, so no pair shares a diagonal. This is a valid solution.

Search strategy: Backtracking/depth-first search can place one queen per column. If a placement conflicts with an earlier queen, undo it and try another row.

Question 4: OLA Cab Booking – Problem-Solving Agent

Suitable agent: A utility-based agent. The customer can compare cab types using preferences such as price, comfort, waiting time, travel time, and sharing. The agent selects the option with the best overall utility while satisfying the customer's requirements.

Inputs: Pickup location, destination, cab types available, estimated fares, estimated arrival/travel times, and customer preferences.

Pseudocode:

```
START
INPUT pickup_location, destination, customer_preferences
GET available_cab_types
best_cab ← NONE
best_utility ← lowest possible value
FOR each cab IN available_cab_types:
    GET fare, pickup_time, travel_time, comfort, sharing_status
    IF cab satisfies customer_preferences:
        utility ← calculate_score(fare, pickup_time, travel_time, comfort, sharing_status)
        IF utility > best_utility:
            best_utility ← utility
            best_cab ← cab
IF best_cab = NONE:
    DISPLAY 'No suitable cab available'
ELSE:
    DISPLAY best_cab and estimated fare/time
    ASK customer to confirm booking
    IF customer confirms: BOOK best_cab
END
```

Output: The most suitable available cab is recommended, and the booking is made only after customer confirmation.

Question 5: Uniform Cost Search (UCS)

Rule: UCS always expands the frontier path with the smallest total accumulated cost g(n).

Edges in the given graph: S→A = 1, S→G = 12, A→B = 3, A→C = 1, B→D = 3, C→D = 1, C→G = 2, D→G = 3.

UCS steps:

Step	Path considered / expanded	Cumulative cost
1	Start at S; frontier: S→A, S→G	A = 1; G = 12
2	Expand A (cost 1)	S→A→C = 2; S→A→B = 4

Step	Path considered / expanded	Cumulative cost
3	Expand C (cost 2)	S→A→C→D = 3; S→A→C→G = 4
4	Expand D (cost 3)	S→A→C→D→G = 6
5	Next lowest goal path is S→A→C→G	4

Path costs:

• S→A→C→G = 1 + 1 + 2 = 4

• S→A→C→D→G = 1 + 1 + 1 + 3 = 6

• S→A→B→D→G = 1 + 3 + 3 + 3 = 10

• Direct S→G = 12

Final answer: The least-cost path is **S** → **A** → **C** → **G**, with total cost **4 cost units**.

End of Solutions